Câu 1: Phân biệt Value Types và Reference Types trong C# về cơ chế lưu trữ vùng nhớ (Stack vs Heap)

1. Phân loại và Định nghĩa

- **Value Types (Kiểu giá trị):** Lưu trữ trực tiếp giá trị thực tế của dữ liệu. Bao gồm các kiểu dữ liệu cơ sở (`int`, `float`, `double`, `bool`, `char`, `byte`), kiểu cấu trúc (`struct`), kiểu liệt kê (`enum`), và kiểu dữ liệu vô hướng (`Tuple`).
- **Reference Types (Kiểu tham chiếu):** Lưu trữ địa chỉ bộ nhớ (con trỏ) trỏ đến vị trí chứa dữ liệu thực tế. Bao gồm lớp (`class`), giao diện (`interface`), ủy nhiệm (`delegate`), chuỗi ký tự (`string`), kiểu đối tượng (`object`), và tất cả các kiểu mảng (`array`).

2. Cơ chế lưu trữ trên Stack và Heap

- **Vùng nhớ Stack (Ngăn xếp):**
  - Được quản lý theo cơ chế **LIFO (Last In, First Out)**, truy xuất cực nhanh.
  - Tự động cấp phát khi biến được khai báo trong hàm/phương thức và tự động giải phóng ngay khi phương thức kết thúc (ra khỏi Scope).
- **Vùng nhớ Heap (Đống):**
  - Vùng nhớ linh hoạt, dung lượng lớn, dùng cho việc cấp phát động.
  - Việc giải phóng bộ nhớ không diễn ra lập tức mà do trình thu gom rác **Garbage Collector (GC)** quản lý tự động theo các chu kỳ quét.

Vị trí lưu trữ chi tiết:

- **Value Types:**
  - Nếu là **biến cục bộ (local variable)** trong phương thức: Toàn bộ giá trị lưu trực tiếp trên **Stack**.
  - Nếu là **thành phần (field/property)** của một lớp (`class`): Giá trị nằm trên **Heap** (đi kèm trong vùng nhớ của đối tượng chứa nó).
- **Reference Types:**
  - Được tách thành 2 phần:
    - **Địa chỉ tham chiếu (Pointer):** Kích thước 4-byte (trên OS 32-bit) hoặc 8-byte (trên OS 64-bit), nằm trên **Stack** (nếu là biến cục bộ).
    - **Dữ liệu thực tế của đối tượng (Actual Object Data):** Cấp phát trên **Heap**.

3. Bảng so sánh tổng hợp

Tiêu chí	Value Types (Kiểu giá trị)	Reference Types (Kiểu tham chiếu)
Vùng nhớ lưu trữ	Stack (biến cục bộ) hoặc Heap (nếu nằm trong Class).	Tham chiếu trên Stack; Dữ liệu thực tế trên Heap.
Cơ chế gán (=)	**Sao chép giá trị (Value copy):** Tạo bản sao hoàn toàn độc lập. Sửa biến này không ảnh hưởng biến kia.	**Sao chép tham chiếu (Reference copy):** Hai biến cùng trỏ vào một vùng nhớ. Sửa qua biến này sẽ đổi dữ liệu của biến kia.
Giá trị mặc định	Không thể nhận `null` (trừ khi dùng `Nullable<T>` / `T?`). Giá trị mặc định là `0`, `false`, `\0`.	Có thể nhận giá trị `null`. Giá trị mặc định khi chưa khởi tạo là `null`.
Quản lý bộ nhớ	Tự động giải phóng khi ra khỏi Scope (trên Stack).	Thu hồi tự động bởi Garbage Collector (GC) khi không còn tham chiếu nào trỏ tới.
Hiệu năng truy xuất	Rất nhanh, không tốn chi phí GC.	Tốn chi phí cấp phát Heap, tra cứu con trỏ và quản lý của GC.

Câu 2: Tính năng Init-only Properties (`init`) trong C# 9/10

1. Phân biệt `init` và `set` thông thường

- **Thuộc tính `set` (Mutable Property):**
  - Cho phép gán hoặc sửa đổi giá trị của thuộc tính **bất kỳ lúc nào** trong suốt vòng đời của đối tượng.
  - Dữ liệu có thể bị thay đổi trạng thái (Mutable) ngoài ý muốn ở bất kỳ đâu trong chương trình.
- **Thuộc tính `init` (Init-only Property):**
  - Được giới thiệu từ C# 9. Chỉ cho phép gán giá trị **duy nhất trong quá trình khởi tạo đối tượng** (thông qua Constructor hoặc cú pháp Object Initializer).

- Sau khi hoàn tất quá trình khởi tạo, thuộc tính trở thành **chỉ đọc (Read-Only)**. Mọi thao tác gán lại giá trị sau đó sẽ bị báo lỗi ngay tại thời điểm biên dịch (**Compile-time Error**).

C#
```
public class UserDto
{
    public string Id { get; init; }   // Chỉ gán khi khởi tạo
    public string Name { get; set; }  // Có thể sửa đổi bất kỳ lúc
nào
}

// Khởi tạo đối tượng
var user = new UserDto
{
    Id = "US-102",  // HỢP LỆ (Trong quá trình khởi tạo)
    Name = "Alice"
};

user.Name = "Bob";  // HỢP LỆ (dùng 'set')
user.Id = "US-103"; // LỖI BIÊN DỊCH: Auto-implemented property
'UserDto.Id' cannot be assigned to -- it is read only
```

2. Trường hợp sử dụng thực tế (Use Cases)

1. **Tạo các Data Transfer Objects (DTO):** Khi truyền nhận dữ liệu giữa các tầng kiến trúc (Controller $\rightarrow$ Service $\rightarrow$ Repository) hoặc nhận response từ API. DTO cần bảo toàn tính toàn vẹn dữ liệu, tránh việc bị chỉnh sửa vô tình trong dòng xử lý.
2. **Thiết kế Value Objects trong Domain-Driven Design (DDD):** Các đối tượng biểu diễn giá trị (như `Money`, `Address`, `GeoLocation`) yêu cầu tính bất biến (Immutability).
3. **Thay thế Constructor quá tải (Overloaded Constructors):** Cho phép khởi tạo đối tượng bất biến linh hoạt với cú pháp Object Initializer `new Obj { Property = value }` mà không cần phải viết quá nhiều Constructor rườm rà.

Câu 3: Phân biệt `virtual` (Lớp cha) và `override` (Lớp con) trong Đa hình (Polymorphism)

1. Khái niệm và Vai trò

- **Phương thức `virtual` (ở Lớp cha - Base Class):**

    - Tuyên bố rằng phương thức này **cho phép** các lớp con ghi đè (redefine) lại logic.
    - **Bắt buộc phải có phần triển khai mặc định (Default Implementation)** trong lớp cha.
- **Phương thức `override` (ở Lớp con - Derived Class):**
    - Tuyên bố định nghĩa lại (ghi đè) logic của phương thức `virtual` đã có từ lớp cha.
    - Bắt buộc phải trùng hoàn toàn về tên, kiểu trả về và danh sách tham số (Method Signature) với phương thức `virtual`.

2. Cơ chế hoạt động tại thời điểm chạy (Runtime / Dynamic Dispatch)

C# sử dụng **Bảng phương thức ảo (Virtual Method Table - vtable)** để thực hiện đa hình động:

- Khi gọi một phương thức thông qua con trỏ tham chiếu kiểu lớp cha nhưng trỏ đến đối tượng thực tế của lớp con (ví dụ: `BaseClass obj = new DerivedClass();`):
    - Trình thực thi C# (CLR) sẽ tra cứu bảng `vtable` của đối tượng thực tế tại Runtime.
    - Nếu lớp con có từ khóa `override`, CLR sẽ thực thi phiên bản phương thức tại **lớp con**.
    - Nếu lớp con không dùng `override` (hoặc dùng từ khóa `new` để ẩn phương thức), CLR sẽ gọi phương thức của **lớp cha**.

3. Ví dụ minh họa và Bảng so sánh

C#
```
public class BankAccount
{
    public virtual double CalculateInterest(double balance)
    {
        return balance * 0.05; // Lãi suất mặc định 5%
    }
}

public class SavingsAccount : BankAccount
{
    public override double CalculateInterest(double balance)
    {
        return balance * 0.08; // Lãi suất riêng cho tiết kiệm 8%
    }
}
```

```
// Thực thi Đa hình (Polymorphism)
BankAccount account = new SavingsAccount();
Console.WriteLine(account.CalculateInterest(1000)); // Kết quả: 80
(Gọi phương thức override ở lớp con)
```

Tiêu chí	virtual (Lớp cha)	override (Lớp con)
Vị trí khai báo	Khai báo tại Lớp cha (Base Class).	Khai báo tại Lớp con (Derived Class).
Mục đích	Mở ra khả năng cho phép thay đổi hành vi ở lớp kế thừa.	Hiện thực hóa hành vi mới cụ thể hơn cho lớp con.
Nội dung xử lý	Cung cấp logic mặc định.	Thay thế hoàn toàn logic mặc định của lớp cha.
Tính bắt buộc	Lớp con không bắt buộc phải override (nếu không ghi đè thì dùng logic mặc định).	Chỉ sử dụng khi lớp cha đã khai báo `virtual`, `abstract`, hoặc `override`.

Câu 4: Lý do thành phần `static` không thể truy xuất thông qua thể hiện (Object Instance)

1. Bản chất về Mức độ sở hữu và Quản lý bộ nhớ (Class-level vs Instance-level)

- **Thành phần `static`** thuộc về **mức độ Lớp (Class level)**. Vùng nhớ của thành phần `static` được cấp phát duy nhất một lần tại vùng nhớ **Type Metadata / Static Heap** khi Class được nạp (Type Loading) vào bộ nhớ. Nó tồn tại xuyên suốt vòng đời ứng dụng và hoàn toàn độc lập với việc có đối tượng nào được tạo ra hay chưa.
- **Thể hiện (Object Instance)** được tạo bằng từ khóa `new` thuộc về **mức độ Đối tượng (Instance level)**. Mỗi instance chiếm một vùng nhớ riêng biệt trên Heap để lưu trạng thái nội bộ của riêng nó.

2. Bản chất kỹ thuật về con trỏ `this`

- Mọi phương thức hoặc thuộc tính thông thường (non-static) khi được gọi qua thể hiện đều nhận ngầm định một tham số là con trỏ `this` (chỉ tới địa chỉ của đối tượng đang gọi).

- Các thành phần `static` **không sở hữu con trỏ** `this`, vì chúng không gắn liền với bất kỳ instance cụ thể nào. Do đó, về mặt cú pháp và bản chất logic, việc gọi `instance.StaticMember` là mâu thuẫn về mặt ngữ nghĩa (semantic mismatch).

3. Thiết kế ngôn ngữ để tránh mơ hồ (Ambiguity) và hiểu nhầm logic

- Nếu cho phép truy cập `instance.StaticMember`, người đọc mã nguồn sẽ lầm tưởng rằng `StaticMember` có dữ liệu riêng biệt trên từng instance đó.
- Việc bắt buộc gọi qua tên lớp (`ClassName.StaticMember`) thể hiện rõ ràng bản chất: **đây là dữ liệu/hành vi dùng chung cho toàn bộ ứng dụng**.

4. Tối ưu hóa hiệu năng biên dịch (Static Binding / Early Binding)

- Khi gọi `ClassName.StaticMember`, trình biên dịch C# có thể liên kết trực tiếp đến địa chỉ ô nhớ static ngay ở giai đoạn biên dịch (**Compile-time / Early Binding**).
- Thao tác này loại bỏ hoàn toàn chi phí runtime như: kiểm tra tham chiếu `null` trên instance hay tra cứu bảng phương thức ảo (`vtable`), giúp tối ưu hóa tốc độ thực thi chương trình.